

## PROJECT AND TEAM INFORMATION

*JEEVANROUTE: Emergency ambulance selection,hospital capability matching and route calculation*

### Student / Team Information

|                           |                                                                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Team Name:<br>Team #      | CYPHERX<br>DSCPP-III-2026-T194                                                                                                                              |
| Team member 1 (Team Lead) | Abhishek kumar,yadav–<br>2510011034<br>abhishubhyadav<br>@gmail.com<br> |
| Team member 2             | Archit, Singh –<br>2510010859<br>architkumarsingh310<br>@gmail.com<br> |
| Team member 3             | Aryan,Singh–<br>2510010893<br>aryansingh1173<br>@gmail.com<br>         |
| Team member 4             | Tushar, Srivastava –<br>2510370201<br>tushar32lko @gmail.com<br>       |

## PROPOSAL DESCRIPTION

### Motivation (1 pt)

- 1. I notice that in areas emergency medical care often faces delays. Healthcare facilities are scarce ambulances are hard to find connectivity is poor. It is difficult to locate the correct hospital.
- 2. When a situation is critical I realize that choosing the hospital may not solve the problem. The patient may need ICU beds, CT scans, specialists, blood or urgent treatment. These delays can greatly reduce the chance that the patient receives care.
- 3. JEEVANROUTE is motivated by the need for an emergency ambulance and hospital routing system that can help reduce these delays. JEEVANROUTE aims to assess the patient's emergency level prioritize cases identify an available ambulance select a hospital based on required facilities and calculate an efficient route.
- 4. From this perspective I see that JEEVANROUTE provides an application of **\*\*Data Structures and Object-Oriented Programming in C++\*\***. Concepts such as priority queues, graphs, lists, classes, objects, encapsulation, inheritance and polymorphism can be applied to solve real-world problems. Thus JEEVANROUTE combines usefulness with learning. JEEVANROUTE also shows how computational techniques can improve coordination between patients healthcare workers, ambulance operators and hospitals, during emergencies.

### State of the Art / Current solution

- 1. **Patient Assessment:** JEEVANROUTE records the patient's details, location and emergency condition.
- 2. **Emergency Prioritization:** JEEVANROUTE sorts patients by seriousness giving the priority to the most critical cases through a priority queue.
- 3. **Ambulance Management:** JEEVANROUTE checks the ambulance's location, availability, capacity and status before dispatching the ambulance.
- 4. **Hospital Selection:** JEEVANROUTE rates hospitals by their qualities. Looks for needed facilities such as ICU beds, CT scans, specialists and blood availability.
- 5. **Route Planning:** JEEVANROUTE represents the road network as a graph enabling the calculation of the route for the ambulance.
- 6. **Ambulance Dispatch:** JEEVANROUTE sends the ambulance to the patient's location after selecting the ambulance and hospital.
- 7. **Hospital Pre-alert:** JEEVANROUTE sends details about the patient and emergency condition to the chosen hospital allowing staff to prepare before the ambulance arrives.
- 8. **Data Management:** JEEVANROUTE stores patient, ambulance, hospital and route information in data structures.
- 9. **OOP Implementation** JEEVANROUTE uses C++ classes and objects applying concepts such, as encapsulation, inheritance and polymorphism.

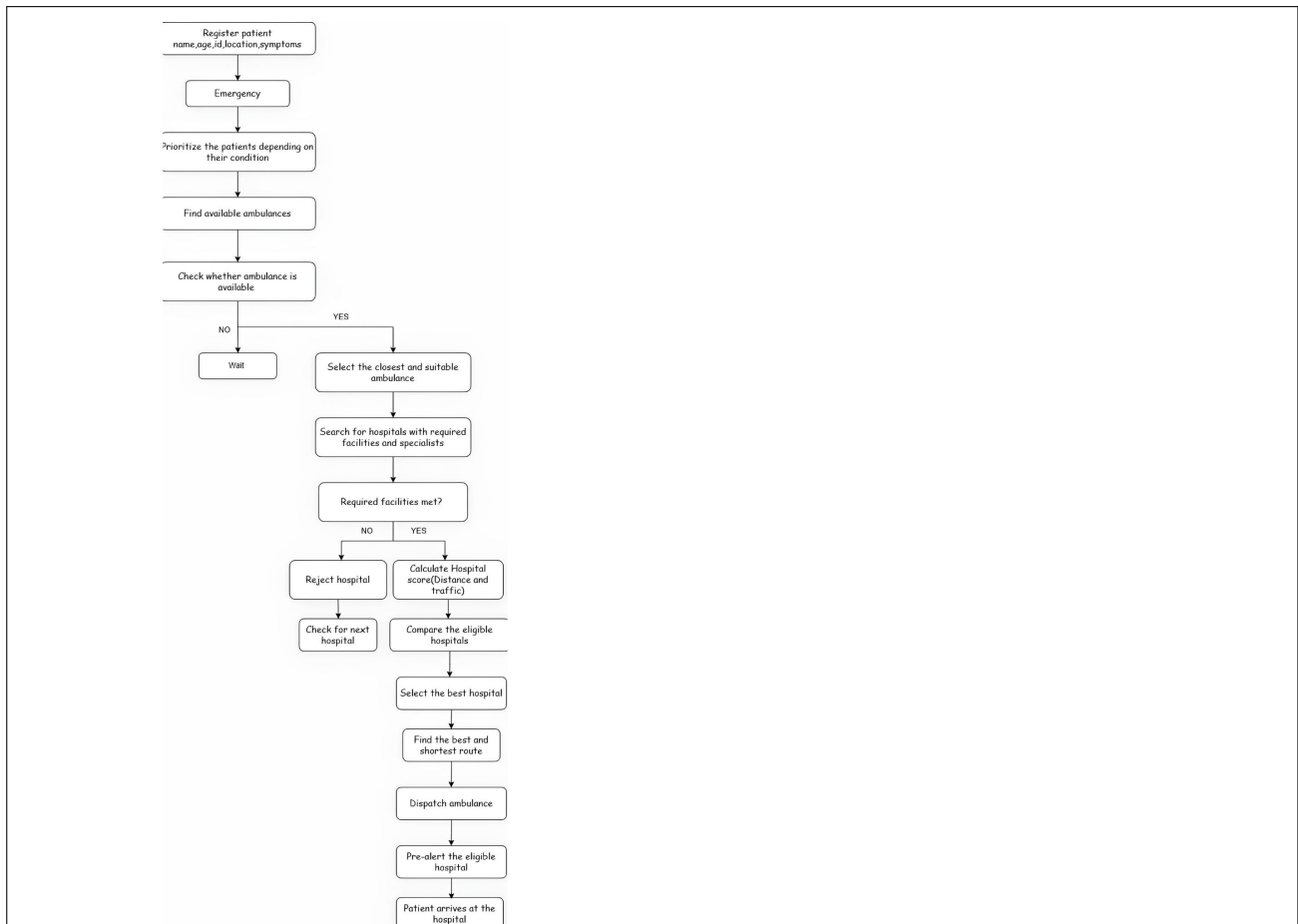
## Project Goals and Milestones

- \* Reduce Emergency Response Time: Aim to shorten the time it takes to get to patients and bring them to hospitals. Faster response means chances for recovery.*
- \* Prioritize Critical Patients: Use a system that decides which patients need help first. The serious the condition the faster they get attention.*
- \* Manage Ambulances Efficiently: Keep a record of where each ambulance is. Know if it is available how people it can carry and what its current status is. This helps assign the ambulance to the right call.*
- \* Select Suitable Hospitals: Choose hospitals that can handle the patient's condition. Look for ICU beds CT scanners, specialists and blood supplies. Make sure the hospital can actually treat the patient.*
- \* Calculate Efficient Routes: Use a map as a network of roads. Find the path by checking distance and how long it takes to travel. The fastest route saves time.*
- \* Improve Emergency Coordination: patients, doctors, ambulance drivers and hospitals in one shared system. Better communication means care.*
- \* Provide Hospital Pre-alerts: Send a warning to the chosen hospital before the ambulance arrives. Let them know about the patient and the type of emergency. This gives the hospital time to prepare.*
- \* Improve Resource Utilization: Match ambulances and hospital resources with what each patient requires. Avoid waste. Keep the system running smoothly.*

## Project Approach

- \* Patient Assessment: Patient Assessment gathers info, about the patient, where they're the type of emergency they face.*
- \* Priority Assignment: Priority Assignment decides which emergencies are serious and places them at the top of the list using a priority queue.*
- \* Ambulance Selection: Ambulance Selection looks for the ambulance that fits the situation.*
- \* Hospital Selection: Hospital Selection chooses a hospital that has the equipment and can handle the patient's needs.*
- \* Route Calculation: Route Calculation uses graphs to find the way to reach the patient.*
- \* Dispatch Pre-: Dispatch Pre- sends the ambulance to the patient. Informs the hospital that an emergency is arriving.*
- \* System Integration: System Integration puts all parts of the system together using C++ data structures and object-oriented programming concepts.*

## System Architecture (High Level Diagram)



## Project Outcome / Deliverables

1. C++. Object-Oriented Programming concepts.
  2. Data Structures concepts. Stacks, Queues, Priority Queues, Lists and Graphs.
  3. Standard C++ Library documentation.
  4. Algorithms, for path and graph traversal.
  - 5.. Articles related to ambulance routing and emergency healthcare systems.
- i).The Rwanda912 paper introduces a **\*\*Destination Decision Support Algorithm (DDSA)\*\***. This algorithm helps ambulance teams pick the hospital when there is an emergency. It looks at the patients condition. It also looks at where the emergency's happening. It checks where the hospital is located. It considers if the hospital is ready. It checks the staff available. It looks at the beds, in the hospital. It checks the equipment there. It also looks at the blood supply. It takes into account the treatments that are needed. The hospital that is suggested can then be. Changed by the ambulance staff or the dispatchers.
- ii).Almalki Aldhahri Aljojo (2025) introduced an ambulance routing system that uses real-time EMS availability. The system picks the routes and medical facilities by looking at the availability of ambulances and hospitals the distance involved and how quickly help can arrive. The main purpose is to cut down on delays and make sure patients get access, to the right kind of care. This aligns with JeevanRoute's method, which also combines distance, traffic conditions, hospital capabilities and real-time updates to improve response efficiency.

## Assumptions

- \* Patient and emergency details are entered correctly.*
- \* Ambulance location, availability, and status are regularly updated.*
- \* Hospital facilities and bed availability are accurate.*
- \* Road distances and routes are predefined.*
- \* GPS/location data is available.*
- \* Suitable ambulances and hospitals are assumed to be available.*
- \* The system supports, but does not replace, professional medical decisions.*

## References

1. *cppreference.com — C++ fundamentals, classes, and file handling (used for shared-file sync).*
2. *GCormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. — Introduction to Algorithms (2nd/3rd edition), MIT Press and McGraw-Hill.*
3. *Reema Thareja — Data Structures Using C, Oxford University Press.*
4. *Francoeur, J. (2002). "Algorithms using Java for Spreadsheet Dependent Cell Recomputation." MITRE Corporation. Available at: [mitre.org/sites/default/files/pdf/francoeur\\_algorithms.pdf](http://mitre.org/sites/default/files/pdf/francoeur_algorithms.pdf)*
5. *GeeksforGeeks — Array Data Structure — Used for implementing the spreadsheet grid. Site → [geeksforgeeks.org/array-data-structure](http://geeksforgeeks.org/array-data-structure)*
6. *Programiz — C++ File Handling — Used for the SYNC/LOAD shared-file collaboration mechanism. Site → [programiz.com/cpp-programming/file-handling](http://programiz.com/cpp-programming/file-handling)*